Zetascope, Inc.

NVIDIA Orin GPU Engineering Plan

8-Week Onboarding and Optimization Roadmap for Track-Before-Detect Flight Software

February 24, 2026

Contents

1 Executive Summary **3**

2 Hardware Context **3**
2.1 NVIDIA Jetson AGX Orin Specifications . . . 3
2.2 Key Architectural Differences . . . 3

3 Prerequisites and Inputs **4**

4 Phase 1: Platform Bring-Up and Baseline (Weeks 1–2) **4**
4.1 Objectives . . . 4
4.2 Week 1: Cross-Compilation and Initial Port . . . 5
4.2.1 Day 1–2: Environment setup . . . 5
4.2.2 Day 3–4: Full-scale benchmark on Orin . . . 5
4.2.3 Day 5: Validation framework . . . 5
4.3 Week 2: Profiling and Bottleneck Identification . . . 6
4.3.1 Day 6–7: Nsight Systems profiling . . . 6
4.3.2 Day 8–9: Nsight Compute kernel profiling . . . 6
4.3.3 Day 10: Bottleneck report . . . 6

5 Phase 2: Memory System Optimization (Weeks 3–4) **7**
5.1 Objectives . . . 7
5.2 Week 3: Unified Memory Architecture . . . 8
5.2.1 Day 11–12: Memory allocation strategy . . . 8
5.2.2 Day 13–14: Memory residency and prefetching . . . 8
5.2.3 Day 15: Frame streaming for memory-constrained scenarios . . . 9
5.3 Week 4: L2 Cache Optimization . . . 9
5.3.1 Day 16–17: Cache working set analysis . . . 9
5.3.2 Day 18–19: Chunk size tuning . . . 9
5.3.3 Day 20: L2 persistence control . . . 10

6 Phase 3: Kernel Optimization (Weeks 5–6) 10
6.1 Objectives . . . 10
6.2 Week 5: Compute Optimizations . . . 11
6.2.1 Day 21–22: Block size and occupancy tuning . . . 11
6.2.2 Day 23–24: Early trajectory rejection . . . 11
6.2.3 Day 25: Loop unrolling and instruction-level optimization . . . 11
6.3 Week 6: Reduced Precision and Advanced Techniques . . . 12
6.3.1 Day 26–27: FP16 frame storage . . . 12
6.3.2 Day 28–29: Warp-level reduction . . . 12
6.3.3 Day 30: Combined benchmarking . . . 13

7 Phase 4: Pipeline Integration and Validation (Weeks 7–8) 13
7.1 Objectives . . . 13
7.2 Week 7: Pipeline Integration . . . 14
7.2.1 Day 31–33: Frame ingest pipeline . . . 14
7.2.2 Day 34–35: Preprocessing integration . . . 14
7.3 Week 8: Thermal Validation and Documentation . . . 15
7.3.1 Day 36–37: Sustained thermal testing . . . 15
7.3.2 Day 38: Power profiling . . . 15
7.3.3 Day 39–40: Documentation and delivery package . . . 15

8 Schedule Summary 16

9 Risk Register 17

10 Expected Outcome 17

1 Executive Summary

This document defines the 8-week engineering plan for a senior GPU engineer joining Zetascope to port and optimize the Track-Before-Detect (TBD) algorithm from development hardware (NVIDIA RTX-class desktop GPUs) to the NVIDIA Jetson AGX Orin platform. The Orin deployment is a **DARPA program requirement** for the TBD2 contract: all delivered code must execute on Orin or equivalent hardware.

The plan is structured in four phases:

1. **Weeks 1–2:** Platform bring-up, baseline port, and profiling
2. **Weeks 3–4:** Memory system optimization for unified memory
3. **Weeks 5–6:** Kernel optimization and compute tuning
4. **Weeks 7–8:** Pipeline integration and flight-mode validation

Each phase concludes with a concrete milestone deliverable. The engineer will work from an existing, validated CUDA codebase running on RTX hardware; the algorithmic correctness is already established.

2 Hardware Context

2.1 NVIDIA Jetson AGX Orin Specifications

Table 1: Orin AGX vs. RTX 4090 comparison.

Parameter	Orin AGX 64GB	RTX 4090	Ratio
Architecture	Ampere (SM 8.7)	Ada Lovelace (SM 8.9)	—
CUDA cores	2,048	16,384	0.125×
GPU clock (max)	1.3 GHz	2.52 GHz	0.52×
FP32 throughput	5.3 TFLOPS	82.6 TFLOPS	0.064×
Memory	64 GB unified	24 GB GDDR6X	2.7× capacity
Memory bandwidth	204.8 GB/s	1,008 GB/s	0.20×
L2 cache	4 MB	96 MB	0.042×
TDP	15–60 W	450 W	0.033–0.13×

2.2 Key Architectural Differences

The performance gap is significant, but three architectural differences dominate the optimization strategy:

1. **Unified memory.** The Orin shares a single physical memory between CPU and GPU. There is no PCIe transfer; instead, the GPU accesses memory directly. However, memory pages must be *resident* on the GPU-accessible fabric for full bandwidth. Unmanaged access patterns cause page faults and catastrophic slowdowns.
2. **L2 cache (4 MB vs. 96 MB).** On the RTX 4090, the entire frame dataset for the interview problem (4 MB) fits in L2. On Orin, *nothing* fits in L2 at operational scale. Every optimization that reduces L2 working set or improves temporal locality has outsized impact on Orin.

3. **Thermal throttling.** The Orin operates under a strict thermal envelope. Sustained full-load kernels will hit thermal limits and clock down. Kernel design must consider sustained throughput, not just peak burst performance.

Optimization intuitions from desktop GPUs do not transfer directly to Orin. Profile on actual Orin hardware early and often. Desktop profiling is necessary but not sufficient.

3 Prerequisites and Inputs

The engineer will receive the following on Day 1:

Table 2: Inputs provided to the GPU engineer.

Item	Description
`tbd_problem.py`	Python reference implementation with data generator, brute-force algorithm, and NumPy-vectorized solver. Defines the algorithm spec and generates validation datasets.
`tbd_solution.cu`	Interview-scale CUDA reference (256×256, 16 frames, 625 velocities). Validated correct on RTX hardware.
`tbd_fullscale.cu`	Full-scale benchmark (4000×2500, 300 frames, $\sim$20K velocities). Self-contained data generation on GPU.
Algorithm specification	Mathematical description of the shift-and-stack test statistic, velocity grid construction, boundary handling, and detection criterion (Sections 2–3 of the interview document).
Orin developer kit	Jetson AGX Orin 64 GB with JetPack SDK installed, accessible via SSH. Provided by Aethero partnership.
Validation datasets	Synthetic datasets with known ground truth at multiple difficulty levels (varying SNR, image size, velocity range). Generated by `tbd_problem.py` with configurable parameters.

4 Phase 1: Platform Bring-Up and Baseline (Weeks 1–2)

4.1 Objectives

1. Get the existing CUDA code compiling and running on Orin
2. Establish baseline performance numbers
3. Identify the dominant bottlenecks via profiling
4. Understand the gap between Orin and RTX performance

4.2 Week 1: Cross-Compilation and Initial Port

4.2.1 Day 1–2: Environment setup

- Install JetPack 6.x SDK on the Orin developer kit (if not pre-installed by Aethero). Verify CUDA 12.x toolkit, cuDNN, and Nsight Systems/Compute are available.
- Set up the build toolchain. The primary change is the target architecture flag:

```
# RTX 4090 (Ada Lovelace)
nvcc -O3 -arch=sm_89 --use_fast_math -o tbd_solution tbd_solution.cu

# Orin (Ampere)
nvcc -O3 -arch=sm_87 --use_fast_math -o tbd_solution tbd_solution.cu
```

- Verify that `tbd_solution.cu` (interview-scale) compiles and produces the correct detection result on Orin. This should require zero code changes beyond the architecture flag.
- Record baseline timing:
  - Interview-scale: expected ∼50–200 ms on Orin (vs. ∼10–30 ms on RTX 4090)
  - Note: this is a sanity check, not a performance target

4.2.2 Day 3–4: Full-scale benchmark on Orin

- Compile and run `tbd_fullscale.cu` on Orin.
- The 12 GB frame buffer fits comfortably in the 64 GB unified memory. However, the `cudaMalloc` in the current code allocates in device-preferred memory, which is the correct behavior on Orin's unified memory architecture.
- Record the full-scale baseline:
  - Expected: ∼10–30 minutes (5× fewer cores, 5× less bandwidth than RTX 4090)
  - This number is the denominator for all subsequent speedup measurements
- If the Orin runs out of memory or crashes, diagnose whether the issue is memory capacity (unlikely at 64 GB) or unified memory page management. Document the failure mode.

4.2.3 Day 5: Validation framework

- Create a test harness that:
  1. Generates synthetic data with known ground truth
  2. Runs the CUDA kernel
  3. Compares detected trajectory against ground truth
  4. Reports PASS/FAIL and timing
- Generate validation datasets at multiple scales:
  - **Small**: 256×256, 16 frames, 625 velocities (seconds on Orin)
  - **Medium**: 1024×1024, 64 frames, 2500 velocities (minutes on Orin)

  - **Full**: 4000×2500, 300 frames, ∼20K velocities (tens of minutes on Orin)

- All three must produce correct detections before proceeding to optimization. Correctness is the immovable constraint.

4.3 Week 2: Profiling and Bottleneck Identification

4.3.1 Day 6–7: Nsight Systems profiling

Run the kernel under Nsight Systems to capture the system-level view:

```
nsys profile --stats=true -o tbd_profile ./tbd_solution
```

- Identify what fraction of time is spent in:
  - Kernel execution
  - Memory allocation / deallocation
  - Host-device synchronization
  - Data generation (if using on-GPU generation)
- On Orin with unified memory, look specifically for:
  - Page fault storms (visible as long stalls in the timeline)
  - CPU–GPU coherence traffic
  - Memory throttling events

4.3.2 Day 8–9: Nsight Compute kernel profiling

Profile the TBD search kernel in detail:

```
ncu --set full -o tbd_kernel_profile ./tbd_solution
```

Capture and document the following metrics:

4.3.3 Day 10: Bottleneck report

Produce a one-page summary document containing:

1. Baseline timing on all three dataset sizes
2. Top-3 bottlenecks ranked by impact
3. Proposed optimization strategy for Phases 2 and 3
4. Estimated speedup target for each optimization

Table 3: Key profiling metrics to capture.

Metric	What It Tells You
Achieved occupancy	Fraction of max warps resident on each SM. Low occupancy ($< 50\%$) suggests register pressure or shared memory limits.
Memory throughput	GB/s achieved vs. theoretical peak (204.8 GB/s). Indicates whether the kernel is memory-bound and how efficiently it uses bandwidth.
L2 hit rate	Fraction of memory reads served from L2 cache. On Orin (4 MB L2), this will be much lower than RTX. Critical for understanding the bandwidth bottleneck.
Warp stall reasons	Where warps are waiting: memory (cache misses), execution (instruction latency), synchronization, or other. Guides optimization priority.
Instruction mix	Ratio of compute (FP32 ops) to memory (load-/store) instructions. Determines if the kernel is compute-bound or memory-bound on Orin.
Register usage per thread	High register count reduces occupancy. Target $\leq$ 48 registers per thread for good occupancy on SM 8.7.

Milestone 1 — End of Week 2

Deliverables:

1. Code compiles and runs correctly on Orin at all three scales
2. Baseline timing recorded (small, medium, full)
3. Nsight profiling report with bottleneck analysis
4. Optimization plan with prioritized action items

Success criterion: Correct detection on all validation datasets. Profiling report reviewed and approved by Zubin.

5 Phase 2: Memory System Optimization (Weeks 3–4)

5.1 Objectives

1. Optimize memory allocation and residency for unified memory
2. Minimize page faults and coherence traffic
3. Maximize effective memory bandwidth on Orin's 204.8 GB/s bus

4. Implement memory prefetching and residency hints

5.2 Week 3: Unified Memory Architecture

5.2.1 Day 11–12: Memory allocation strategy

The Orin's unified memory means CPU and GPU share the same physical DRAM. The current code uses `cudaMalloc`, which allocates device-preferred memory. On Orin, this is correct but may not be optimal for all buffers.

Evaluate three allocation strategies and benchmark each:

1. `cudaMalloc` (current): Device-resident allocation. GPU accesses are fast; CPU access triggers migration. Appropriate for the frame buffer (GPU-only access).

2. `cudaMallocManaged` with prefetch: Managed memory that can be explicitly prefetched to GPU before kernel launch. Useful for output buffers that the CPU needs to read after the kernel completes.

```
// Allocate managed memory
float* best_stats;
cudaMallocManaged(&best_stats, n_pixels * sizeof(float));

// Prefetch to GPU before kernel launch
cudaMemPrefetchAsync(best_stats, n_pixels * sizeof(float),
                     deviceId, stream);

// After kernel, prefetch back to CPU for reduction
cudaMemPrefetchAsync(best_stats, n_pixels * sizeof(float),
                     cudaCpuDeviceId, stream);
```

3. `cudaHostAlloc` with mapped access: Pinned host memory accessible from GPU via zero-copy. Lower bandwidth but avoids explicit transfers. Evaluate for output buffers only.

5.2.2 Day 13–14: Memory residency and prefetching

For the frame buffer (12 GB at full scale), implement explicit memory management:

- **Memory advise hints:** Use `cudaMemAdvise` to inform the runtime about access patterns:

```
// Frame data: read-only on GPU, never accessed by CPU during search
cudaMemAdvise(d_frames, frame_bytes,
              cudaMemAdviseSetReadMostly, deviceId);
cudaMemAdvise(d_frames, frame_bytes,
              cudaMemAdviseSetPreferredLocation, deviceId);

// Prefetch the entire frame buffer before starting search
cudaMemPrefetchAsync(d_frames, frame_bytes, deviceId, stream);
cudaStreamSynchronize(stream);
```

- **Measure the impact:** Profile with and without advise hints. On Orin, the difference between "GPU accesses un-advised managed memory" and "GPU accesses properly prefetched memory" can be 10× or more due to page fault handling overhead.

5.2.3 Day 15: Frame streaming for memory-constrained scenarios

Although the Orin 64 GB can hold the full 12 GB frame buffer, future operational scenarios may involve larger datasets or shared memory with other processes. Implement a frame-streaming mode:

- Divide frames into temporal chunks (e.g., 50 frames at a time)
- Use double-buffered prefetching: while the kernel processes chunk k, prefetch chunk $k+1$
- This requires modifying the kernel to accept a frame offset parameter and accumulate partial statistics across chunks

Design Decision

Frame streaming adds complexity and is not needed for current data sizes on the 64 GB Orin. Implement it as a *build option* (compile flag) so it is available but not in the default path. This future-proofs the code without adding runtime overhead.

5.3 Week 4: L2 Cache Optimization

The Orin's 4 MB L2 cache is the most significant performance constraint. A single frame is 40 MB at full scale — 10× larger than L2. This means *every frame access is an L2 miss at operational scale.*

5.3.1 Day 16–17: Cache working set analysis

Instrument the kernel to understand the L2 access pattern:

- For a fixed velocity (v_x, v_y) and frame t, all threads in a warp read pixels at addresses base + $(y_0+v_y t)\cdot N_x+(x_0+v_x t)$. Adjacent threads (adjacent x_0) read adjacent addresses — **coalesced** for small $v_x t$.
- The L2 working set per frame access is $\sim N_x \times$ chunk_rows $\times$ 4 bytes. For a 100-row chunk: $4000 \times 100 \times 4 = 1.6$ MB, which fits in L2. This means the row-chunk strategy from the full-scale benchmark is not just a TDR workaround — it is a **critical L2 optimization on Orin**.

5.3.2 Day 18–19: Chunk size tuning

Systematically vary the row-chunk size and measure throughput:

Table 4: Chunk size vs. L2 working set on Orin.

Chunk rows	L2 working set/frame	Fits in L2?	Expected perf.
25	0.4 MB	Yes, with room	Underutilized GPU
50	0.8 MB	Yes	Good balance
100	1.6 MB	Borderline	Near optimal
250	4.0 MB	Exactly fills L2	Thrashing begins
500	8.0 MB	No (2× L2)	Significant misses

The optimal chunk size will be the largest that keeps the per-frame working set within L2. Benchmark each setting on Orin hardware and select empirically.

5.3.3 Day 20: L2 persistence control

CUDA 11.0+ supports L2 cache persistence controls via `cudaAccessPolicyWindow`. On Orin, this allows reserving a portion of L2 for the frame data:

```
cudaAccessPolicyWindow policy = {};
policy.base_ptr   = (void*)d_frames;
policy.num_bytes  = chunk_frame_bytes;  // Size of current chunk's frames
policy.hitRatio   = 1.0f;               // Keep ALL lines persistent
policy.hitProp    = cudaAccessPropertyPersisting;
policy.missProp   = cudaAccessPropertyStreaming;

cudaDeviceProp prop;
cudaGetDeviceProperties(&prop, 0);
size_t l2_reserve = min(chunk_frame_bytes,
                        (size_t)(prop.l2CacheSize * 0.75));
cudaDeviceSetLimit(cudaLimitPersistingL2CacheSize, l2_reserve);

// Apply policy to the stream
cudaStreamSetAttribute(stream,
    cudaStreamAttributeAccessPolicyWindow, &policy);
```

Benchmark with and without persistence control. The expected benefit is highest when the working set is close to L2 size (the 100-row chunk regime).

Milestone 2 — End of Week 4

Deliverables:

1. Memory allocation strategy documented and benchmarked (3 strategies compared)
2. Prefetching and memory advise hints implemented
3. Optimal chunk size determined empirically for Orin
4. L2 persistence control evaluated
5. Frame streaming mode implemented as build option
6. Performance improvement documented: expected 1.5–3× over Phase 1 baseline

Success criterion: Full-scale benchmark runs 1.5×+ faster than Week 2 baseline. Correctness preserved on all validation datasets.

6 Phase 3: Kernel Optimization (Weeks 5–6)

6.1 Objectives

1. Maximize compute throughput per watt on Orin
2. Reduce total pixel reads through algorithmic shortcuts
3. Tune occupancy and instruction throughput
4. Evaluate reduced-precision arithmetic

6.2 Week 5: Compute Optimizations

6.2.1 Day 21–22: Block size and occupancy tuning

The current kernel uses $16 \times 16 = 256$ threads per block. On Orin's SM 8.7 with 48 KB shared memory and 65,536 registers per SM, the occupancy depends on register usage per thread.

- Profile actual register usage:

```
nvcc -O3 -arch=sm_87 --use_fast_math --ptxas-options=-v \
     -o tbd_solution tbd_solution.cu
```

  The `--ptxas-options=-v` flag reports registers per thread.

- If register usage exceeds 48 (limiting occupancy to 1 block per SM), consider:
  - `__launch_bounds__(256, 2)` to hint the compiler to target 2 blocks per SM
  - Restructuring the inner loop to reduce live variables
  - Explicitly spilling the best-velocity tracking to local memory
- Benchmark block sizes: 8×8, 16×8, 16×16, 32×8, 32×4. On Orin with fewer SMs (16 SMs for 2048 cores), block size affects SM utilization differently than on a 128-SM RTX 4090.

6.2.2 Day 23–24: Early trajectory rejection

A significant fraction of trajectories are invalid (exit the image before reaching frame $N - 1$). The current code discovers this inside the inner loop via the boundary check. We can skip many velocity hypotheses *analytically* before entering the loop:

```
// For starting position (x0, y0), check if velocity (vx, vy)
// keeps the trajectory in-bounds at t = N_FRAMES - 1
int end_px = __float2int_rn((float)x0 + vx * (float)(nf - 1));
int end_py = __float2int_rn((float)y0 + vy * (float)(nf - 1));

// If endpoint is out of bounds, skip entirely (no inner loop)
if (end_px < 0 || end_px >= nx || end_py < 0 || end_py >= ny)
    continue;
```

Impact Analysis

For a 4000×2500 image with velocities up to ± 3.5 pixels/frame and 300 frames, the maximum displacement is $3.5 \times 299 = 1046.5$ pixels. Any pixel within 1047 pixels of the boundary will have many invalid velocity hypotheses. The fraction of work saved depends on the velocity range relative to the image size.

At the edges, the endpoint check alone eliminates the majority of velocity pairs. For a corner pixel, roughly 75% of velocities are invalid. Averaged over all pixels, the expected work reduction is ∼20–40% for these parameters.

6.2.3 Day 25: Loop unrolling and instruction-level optimization

- The inner frame loop ($t = 0 \ldots N - 1$) is a candidate for partial unrolling. With $N = 300$ frames, full unrolling is impractical (register pressure), but unrolling by 4 or 8 hides instruction latency:

```
#pragma unroll 4
for (int t = 0; t < nf; t++) {
    // ... pixel read and accumulate
}
```

- Replace the `int(round(...))` pattern with `__float2int_rn()` (already in the reference code). Verify the compiler is using the hardware rounding instruction and not emitting a software `roundf()` call.
- Evaluate `--use_fast_math` impact: this flag enables `__fmul_rn` → fused multiply-add optimizations. Verify it does not change detection results (it should not for integer-rounded coordinate computation, but verify).

6.3 Week 6: Reduced Precision and Advanced Techniques

6.3.1 Day 26–27: FP16 frame storage

The frame data is generated as FP32 (4 bytes/pixel), but Poisson noise with $\lambda = 100$ produces values in the range $[50, 170]$ with high probability. FP16 (`half`) has sufficient range and precision for this data, and halving the memory footprint doubles effective bandwidth.

- Convert the frame buffer from FP32 to FP16 on ingest:

```
#include <cuda_fp16.h>

// Convert FP32 frames to FP16 (one-time preprocessing)
__global__ void convert_fp32_to_fp16(const float* src, half* dst, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) dst[i] = __float2half(src[i]);
}

// In the search kernel, read FP16, accumulate in FP32
stat += __half2float(__ldg(&frames_fp16[offset]));
```

- Frame buffer size: $10\text{M} \times 300 \times 2 = 6\,\text{GB}$ (down from 12 GB). This improves bandwidth utilization by 2× and also improves L2 hit rates since twice as many pixel values fit in the 4 MB cache.
- **Validation:** Run the full detection pipeline with FP16 frames and verify that the detection result is unchanged. The accumulator remains FP32, so numerical drift should be negligible.

6.3.2 Day 28–29: Warp-level reduction

Replace the current host-side global reduction with a three-stage GPU reduction:

1. **Warp-level:** Use `__shfl_down_sync()` to reduce within each 32-thread warp without shared memory:

```
// Warp-level max reduction
for (int offset = 16; offset > 0; offset >>= 1) {
    float other_stat = __shfl_down_sync(0xFFFFFFFF, local_best, offset);
    int   other_idx  = __shfl_down_sync(0xFFFFFFFF, local_idx, offset);
    if (other_stat > local_best) {
```

```
        local_best = other_stat;
        local_idx  = other_idx;
    }
}
```

2. **Block-level:** First warp of each block reduces across warp results using shared memory.
3. **Grid-level:** A second kernel (or atomic operations) reduces across block results.

This eliminates the 40 MB device-to-host copy for the per-pixel results. On Orin, where CPU–GPU coherence traffic is expensive, this has meaningful impact.

6.3.3 Day 30: Combined benchmarking

Run the full-scale benchmark with all Phase 3 optimizations enabled, both individually and combined:

Table 5: Expected impact of Phase 3 optimizations (cumulative).

Optimization	Expected Speedup	Notes
Early trajectory rejection	1.2–1.4×	Depends on edge fraction
Block size tuning	1.1–1.3×	Occupancy improvement
FP16 frame storage	1.5–1.8×	Bandwidth reduction
Loop unrolling	1.05–1.1×	Instruction latency hiding
GPU reduction	1.05–1.1×	Eliminates host copy
Combined (est.)	**2–4×**	Over Phase 2 result

Milestone 3 — End of Week 6

Deliverables:

1. Optimized kernel with early rejection, FP16 frames, tuned block size, and GPU-side reduction
2. Each optimization benchmarked individually and in combination
3. Occupancy and instruction mix profiled with Nsight Compute
4. Correctness verified on all validation datasets
5. Performance improvement: expected 3–8× over original baseline

Success criterion: Full-scale benchmark runs at least 3× faster than Week 2 baseline on Orin. Detection correctness preserved.

7 Phase 4: Pipeline Integration and Validation (Weeks 7–8)

7.1 Objectives

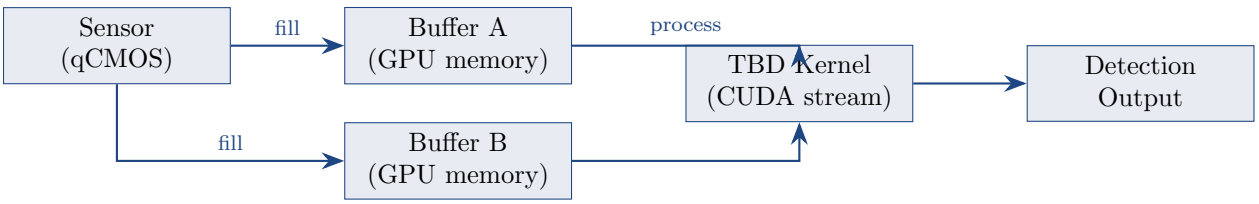
1. Integrate the optimized kernel into the full processing pipeline

2. Implement real-time-compatible frame ingest
3. Validate under sustained thermal conditions
4. Document the system for DARPA milestone delivery

7.2 Week 7: Pipeline Integration

7.2.1 Day 31–33: Frame ingest pipeline

In the operational system, frames arrive from the sensor in real time. The TBD kernel must consume frames as they arrive, not from a pre-allocated buffer. Implement a double-buffered ingest architecture:



- **Buffer A** receives frames while **Buffer B** is being processed by the TBD kernel (and vice versa)
- Use CUDA streams to overlap data ingest (CPU→GPU copy or DMA from sensor) with kernel execution
- On Orin's unified memory, the "copy" may be a simple pointer swap if the sensor driver writes directly to managed memory

7.2.2 Day 34–35: Preprocessing integration

The raw sensor frames require preprocessing before TBD:

1. **Dark frame subtraction:** $I_t \leftarrow I_t - D$ where D is a pre-computed dark frame. This is a trivial element-wise CUDA kernel, but it must run in the same stream as the TBD kernel to avoid synchronization stalls.
2. **Cosmic ray rejection:** Flag pixels exceeding a threshold (e.g., $> 5\sigma$) as invalid. Replace with local median or background estimate. This must be a separate kernel preceding TBD.
3. **Star subtraction:** Remove known star signals using a catalog-based model. This requires catalog lookup and PSF-matched subtraction. Implement as a kernel or defer to a separate module depending on complexity.

All preprocessing kernels should be fused where possible (e.g., dark subtraction + cosmic ray flagging in a single kernel pass) to minimize memory traffic.

7.3 Week 8: Thermal Validation and Documentation

7.3.1 Day 36–37: Sustained thermal testing

- Configure the Orin in its target power mode:

```
# Set power mode (example: 50W for space-sim conditions)
sudo nvpmodel -m 0          # Max performance (60W)
sudo nvpmodel -m 3          # 30W mode (closer to space thermal)
sudo jetson_clocks          # Lock clocks for reproducible measurements
```

- Run the full-scale benchmark continuously for 30+ minutes:
  - Monitor GPU temperature: `tegrastats --interval 1000`
  - Record throughput over time
  - Identify if/when thermal throttling occurs
  - Determine the **sustained** throughput (steady-state after thermal equilibrium) vs. the burst throughput (first 1–2 minutes)
- The sustained throughput number is the one that matters for DARPA milestone reporting. Peak burst numbers are misleading for continuous operational scenarios.

7.3.2 Day 38: Power profiling

- Measure actual power draw during kernel execution:

```
# Read power sensors on Orin
cat /sys/bus/i2c/drivers/ina3221/*/hwmon/hwmon*/power*_input
```

- Compute the energy efficiency metric:

$$\eta = \frac{\text{trajectories evaluated}}{\text{joules consumed}} \quad [\text{trajectories/J}] \tag{1}$$

- Compare across power modes. The optimal operating point may not be the highest-performance mode if the trajectories-per-joule efficiency peaks at a lower clock.

7.3.3 Day 39–40: Documentation and delivery package

Produce the following documentation for DARPA milestone delivery and internal use:

1. **Technical report:** Architecture description, optimization decisions and rationale, performance results on Orin, comparison to desktop GPU, scaling analysis.
2. **Build and deployment guide:** Step-by-step instructions to compile and run on a fresh Orin developer kit. Include JetPack version requirements, build flags, and test procedures.
3. **Performance datasheet:** One-page summary suitable for inclusion in DARPA milestone reports:
   - Throughput (trajectories/second, pixels/second)
   - Latency (wall-clock time for full-scale search)

- Power draw (watts, joules per search)
- Sustained vs. burst performance

4. **Validation report:** Detection results on all test datasets, including false alarm rate characterization, detection probability at varying SNR levels, and comparison to Python reference.
5. **Source code:** Clean, documented, version-controlled repository with build system, test suite, and CI configuration.

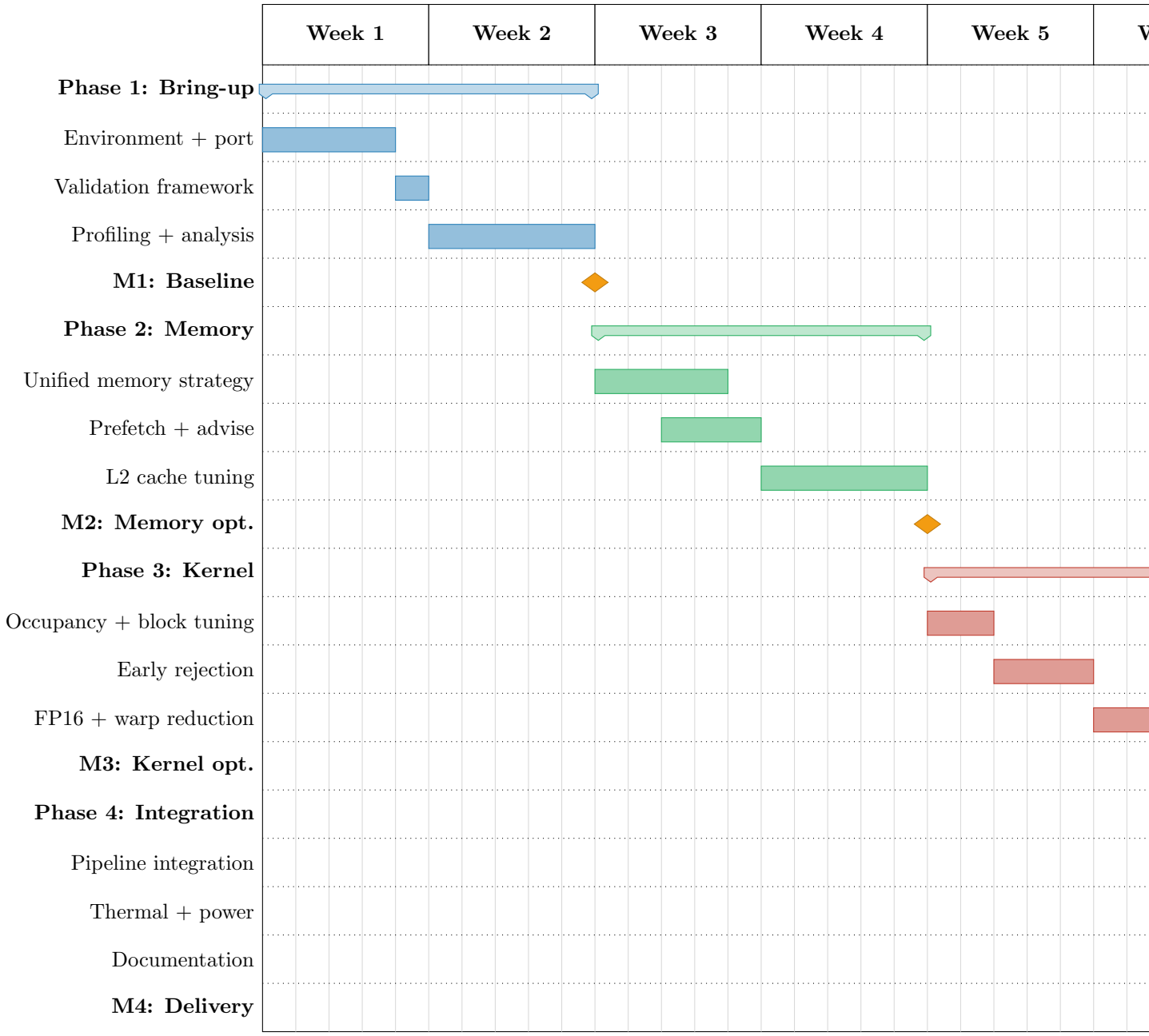
Milestone 4 — End of Week 8

Deliverables:

1. Complete TBD pipeline running on Orin: frame ingest → preprocessing → TBD search → detection output
2. Sustained thermal performance characterized
3. Power efficiency measured across operating modes
4. Full documentation package for DARPA milestone delivery
5. Clean source repository with build system and test suite

Success criterion: End-to-end pipeline runs correctly on Orin under sustained thermal load. Documentation sufficient for DARPA review.

8 Schedule Summary



9 Risk Register

10 Expected Outcome

At the end of 8 weeks, the following performance envelope is expected for the brute-force TBD kernel on the NVIDIA Jetson AGX Orin 64 GB:

These numbers are for the **brute-force** algorithm at full operational scale (10 MP × 20K velocities × 300 frames). The brute-force kernel is the correctness baseline, not the operational target. The temporal-velocity pyramid and other algorithmic accelerations (developed separately by the algorithm lead) provide 100–1000× additional reduction, bringing the operational system into the real-time regime. The GPU engineer's optimized brute-force kernel serves as:

1. The correctness reference against which all accelerated algorithms are validated
2. The inner-loop kernel that the accelerated algorithms invoke on reduced search spaces
3. The worst-case fallback for regions where acceleration assumptions break down

Zetascope, Inc. — Internal Engineering Document
Do not distribute outside the organization.

Table 6: Key risks and mitigations.

Risk	Likelihood	Impact	Mitigation
Orin devkit hardware delay	Medium	Blocks all work until hardware arrives	Begin Phase 1 using Jetson Orin NX (pin-compatible, lower performance) or NVIDIA's cloud Jetson instances.
Thermal throttling worse than expected	Medium	Sustained throughput 50–70% of burst	Characterize early in Week 2. Adjust power mode and kernel duty cycle. Consider active cooling modifications.
FP16 precision insufficient	Low	Detection results change vs. FP32 reference	Maintain FP32 fallback path. Validate FP16 at each SNR level. Use FP16 storage with FP32 accumulation (not pure FP16 compute).
Unified memory performance pathological	Low	Page fault storms degrade bandwidth by 10×+	Explicit prefetch on Day 1. Fall back to `cudaMalloc` with explicit copies if needed.
Algorithm spec changes during 8 weeks	Medium	Rework required; schedule slips	Freeze algorithm spec before engineer starts. Changes queued for Phase 5 (post initial delivery).
Engineer unfamiliar with Jetson platform	Medium	Weeks 1–2 take longer than planned	Budget 2 full weeks for bring-up. Provide Aethero BSP documentation on Day 1. Accept that this is the ramp-up cost.

Table 7: Projected performance at end of Week 8.

Metric	Week 2 Baseline	Week 8 Target
Full-scale search time (burst)	10–30 min	2–5 min
Full-scale search time (sustained)	15–40 min	3–8 min
Power draw	60 W (max)	30–50 W (tuned)
Energy per search	∼50 kJ	∼10 kJ
Memory footprint	12 GB (FP32)	6 GB (FP16)